

LEGALEASE

2. Requirement Analysis Phase

LegalEase – Functional and Non-Functional Requirements

PROJECT TEAM

- Nikkath Khatoon (Team Leader)
- Nowfel.M
- Snehan.P
- Joel.J
- Arfana Begum

1. Reference Basis

Prepared using the supplied Periyar University-style reference format and the supplied LegalEase project document. The reference structure covers project title/description, use cases, architecture, prerequisites, objectives, workflow, milestones, testing, deployment, results, limitations, future enhancements and appendix. The LegalEase source describes an AI-powered legal document generator using Streamlit, FastAPI, Gemini integration and TXT/DOCX/PDF export.

2. User Requirements

- Select legal-document type
- Enter parties
- Enter terms
- Enter effective date
- Generate document
- Preview and edit output
- Download generated file

3. Functional Requirements

- FR-01 Accept document type
- FR-02 Accept parties
- FR-03 Accept terms
- FR-04 Accept dates
- FR-05 Send request to backend
- FR-06 Generate through Gemini
- FR-07 Display result
- FR-08 Edit result
- FR-09 Export TXT/DOCX/PDF

4. Non-Functional Requirements

- Usability
- Performance
- Security
- Maintainability
- Reliability
- Scalability

5. Technical Requirements

- Python 3.10+

- FastAPI
- Uvicorn
- Streamlit
- Google Generative AI SDK
- python-docx
- FPDF
- Pillow
- requests
- python-dotenv

6. API Requirement

The supplied LegalEase document specifies a POST /generate endpoint using a Pydantic DocumentRequest and GeminiDocumentGenerator.

7. Input Fields

- document_type
- parties
- terms
- dates